Understanding the Event-Driven .NET + Angular Project

1. Goal of the Project

This project demonstrates Event-Driven Architecture (EDA) using .NET services and a message broker. The system allows one service to publish an event when something happens (for example, an order is created). Other services listen for that event and react independently.

2. Key Technologies Used

.NET: Backend platform used to build services.

ASP.NET Core: Used to expose HTTP APIs.

MassTransit: A library that simplifies working with message brokers.

RabbitMQ: The message broker responsible for delivering events between services.

Angular: Frontend application that calls the Order API.

3. High-Level Architecture

The system is composed of multiple small services. Each service has a single responsibility.

OrderService: Handles creation of orders.

InventoryService: Reduces stock when an order is created.

NotificationService: Sends a notification when an order is created.

RabbitMQ sits in the middle and routes messages between services.

4. Event-Driven Flow

Step 1: A user sends a request to create an order.

Step 2: OrderService processes the request.

Step 3: OrderService publishes an OrderCreatedEvent.

Step 4: RabbitMQ receives the event.

Step 5: InventoryService and NotificationService consume the event.

Step 6: Each service performs its own action.

5. Why Event-Driven Architecture Is Used

Services are loosely coupled.

Services do not need to know about each other.

New services can subscribe to events without modifying existing services.

The system scales more easily.

6. OrderService Responsibilities

Expose an API endpoint to create orders.

Execute business logic.

Publish an OrderCreatedEvent using MassTransit.

This service follows Clean Architecture with Domain, Application, Infrastructure, and API layers.

7. InventoryService Responsibilities

This service subscribes to OrderCreatedEvent.

When the event arrives, it reduces the product stock.

It does not need to know who created the order.

8. NotificationService Responsibilities

This service also subscribes to OrderCreatedEvent.

When it receives the event, it sends a notification (simulated using a console log).

9. Role of RabbitMQ

RabbitMQ acts as the message broker.

It receives events from publishers and distributes them to all subscribed consumers.

This allows multiple services to react to the same event.

10. Why the System May Feel Confusing at First

Unlike traditional architectures where services call each other directly, EDA relies on asynchronous communication. Instead of calling another service, a service publishes an event and does not know who will react to it.

11. How to Test the System

1. Start RabbitMQ.

2. Run OrderService.

3. Run InventoryService.

4. Run NotificationService.

5. Send POST /orders request.

6. Observe logs in InventoryService and NotificationService.

12. Key Idea to Remember

In Event-Driven Architecture:

Services publish events when something happens.

Other services subscribe and react to those events.

This allows systems to grow without tight dependencies.